

---

## SERVLET CREATION IN JAVA

---

### 1. SERVLET HIERARCHY

---

There are 3 main ways to create a Servlet:

Servlet Interface

|

v

GenericServlet

|

v

HttpServlet

---

### A) IMPLEMENTING Servlet INTERFACE

---

Servlet is the parent interface.

It defines basic Servlet life-cycle methods such as:

```
init()  
service()  
destroy()
```

Example:

```
public class MyServlet implements Servlet {  
    // implement Servlet methods  
}
```

This approach is rarely used directly because we have easier options available.

---

## B) EXTENDING GenericServlet

---

GenericServlet implements the Servlet interface.

It is used when you want to create a protocol-independent Servlet.

Example:

```
public class MyServlet extends GenericServlet {
```

```
// Servlet logic  
}
```

It does not specifically focus on HTTP.

---

### C) EXTENDING HttpServlet

---

HttpServlet extends GenericServlet.

This is the MOST COMMON and PREFERRED way to create a Servlet for web applications.

It provides HTTP-specific methods such as:

```
doGet()  
doPost()  
doPut()  
doDelete()
```

Example:

```
public class MyServlet extends HttpServlet {
```

```
}
```

For normal web applications, we generally use:

HttpServlet

=====

## 2. CREATING A SERVLET IN ECLIPSE

=====

### STEP 1:

Create a Dynamic Web Project in Eclipse.

Make sure the option to generate:

web.xml

(Deployment Descriptor) is enabled.

### STEP 2:

Create a Java class inside the source folder.

Example:

```
package in.sp.backend;

import jakarta.servlet.http.HttpServlet;

public class MyServlet extends HttpServlet {

}
```

STEP 3:

Override an HTTP method.

For example:

```
doGet()
```

The doGet() method receives two important objects:

HttpServletRequest

HttpServletResponse

Example:

```
@Override
protected void doGet(HttpServletRequest request,
                      HttpServletResponse response) {
```

}

=====

### 3. WHAT IS web.xml?

=====

web.xml is called the:

Deployment Descriptor

It contains configuration information about the web application.

It tells the server:

URL

|

v

Servlet Name

|

v

Java Servlet Class

=====

#### 4. SERVLET MAPPING IN web.xml

=====

We use two important tags:

<servlet>

<servlet-mapping>

-----

<servlet>

-----

This connects a servlet name with the actual Java class.

Example:

<servlet>

<servlet-name>MyServlet</servlet-name>

<servlet-class>in.sp.backend.MyServlet</servlet-class>

</servlet>

Here:

MyServlet

|

v

Servlet name

in.sp.backend.MyServlet

|

v

Actual Java class

---

<servlet-mapping>

---

This connects the servlet name with a URL.

Example:

```
<servlet-mapping>  
  <servlet-name>MyServlet</servlet-name>  
  <url-pattern>/aaa</url-pattern>  
</servlet-mapping>
```

Here:



/aaa

|

v

MyServlet

|

v

in.sp.backend.MyServlet

IMPORTANT:

The servlet-name must be exactly the same in both places.

Example:

<servlet-name>MyServlet</servlet-name>

must match:

<servlet-name>MyServlet</servlet-name>

=====

5. COMPLETE web.xml EXAMPLE

=====

Example:

```
<web-app>

  <servlet>
    <servlet-name>MyServlet</servlet-name>
    <servlet-class>
      in.sp.backend.MyServlet
    </servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>MyServlet</servlet-name>
    <url-pattern>/aaa</url-pattern>
  </servlet-mapping>

</web-app>
```

The flow is:

```
Browser
|
| /aaa
v
```

Tomcat

|

v

servlet-mapping

|

| MyServlet

v

servlet

|

| in.sp.backend.MyServlet

v

MyServlet.java

|

v

doGet()

=====

## 6. HOW THE REQUEST WORKS

=====

Suppose the URL is:

<http://localhost:8080/ServletLearning/aaa>

The process is:

Browser sends request

|

v

Tomcat

|

v

Checks web.xml

|

v

Finds /aaa

|

v

Finds MyServlet

|

v

Finds MyServlet.java

|

v

doGet()

|

v

Sends response

|

v

Browser

=====

## 7. System.out.println() VS response.getWriter()

=====

If you write:

```
System.out.println("Hello");
```

the output goes to:

Eclipse Console

It does NOT directly appear in the browser.

-----

To display output in the browser:

-----

Use:

```
PrintWriter out = response.getWriter();
```

Then:

```
out.print("Hello");
```

Example:

```
@Override
protected void doGet(HttpServletRequest request,
                      HttpServletResponse response)
    throws IOException {

    PrintWriter out = response.getWriter();

    out.print("Hello from Servlet");

}
```

The result:

Browser

-----

Hello from Servlet

-----

=====

## 8. IMPORTANT POINTS TO REMEMBER

=====

1. Servlet is the parent interface.
2. GenericServlet implements Servlet.
3. HttpServlet extends GenericServlet.
4. HttpServlet is the most commonly used approach.
5. doGet() is used for handling HTTP GET requests.
6. doPost() is used for handling HTTP POST requests.
7. web.xml is the Deployment Descriptor.
8. <servlet> connects a servlet name to a Java class.
9. <servlet-mapping> connects a URL to a servlet name.
10. The servlet-name must match in both tags.
11. System.out.println() prints to the Eclipse console.

12. response.getWriter() is used to send output to the browser.

=====

## QUICK REVISION

=====

### Servlet

|

+---- Servlet Interface

|

+---- GenericServlet

|

+---- HttpServlet <-- Usually used

### Request Flow:

#### Browser

|

v

#### URL

|

v

#### web.xml

|



v

Servlet Mapping

|

v

Servlet Class

|

v

doGet()/doPost()

|

v

Response

|

v

Browser

=====

Video link: <https://youtu.be/fa2nWRMnpr8?si=2Na1Rxwlf-safmYV>